İSTANBUL OKAN ÜNİVERSİTESİ | MESLEK YÜKSEKOKULU

MMOT225: Bulut Bilişim

Dönem Sonu Projesi Yönergesi

Dr. Öğr. Üyesi Hasan Oğuz

DİKKAT! PROJE TESLİM KURALLARI

Teslim sadece O'Learn üzerinden yapılacaktır.

Projenin değerlendirilebilmesi için laboratuvarda **canlı olarak çalıştırılıp gösterilmesi** ve **sınav tutanağına imza atılması** zorunludur.

Çalışmayan, açıklanamayan veya kopya olduğu tespit edilen projeler **değerlendirme dışı bırakılacaktır.**

1. Projenin Amacı ve Senaryo

Dönem boyunca işlenen sanallaştırma, Linux ortamı, Docker, konteyner mantığı ve bulut bilişim kavramlarını pratikte uygulamak amacıyla bu proje hazırlanmıştır. Öğrenciden, tek bir imajdan birden fazla müşteri için izole biçimde çalışabilen basit bir **SaaS MVP (Minimum Viable Product)** sistemi geliştirmesi beklenmektedir.

Senaryo: Bir bulut hizmet sağlayıcısı olduğunuzu varsayın. Aynı Flask tabanlı müşteri paneli uygulamasını farklı firmalara hizmet verecek biçimde Docker ile paketleyecek, sonra aynı imajdan birden fazla container üretip bunları farklı portlar üzerinden yayınlayacaksınız. Böylece aynı uygulama mantığı, farklı müşterilere özelleştirilmiş şekilde sunulacaktır.

2. Projenin Teknik Gereksinimleri

app.py dosyası için zorunlu gereksinimler:

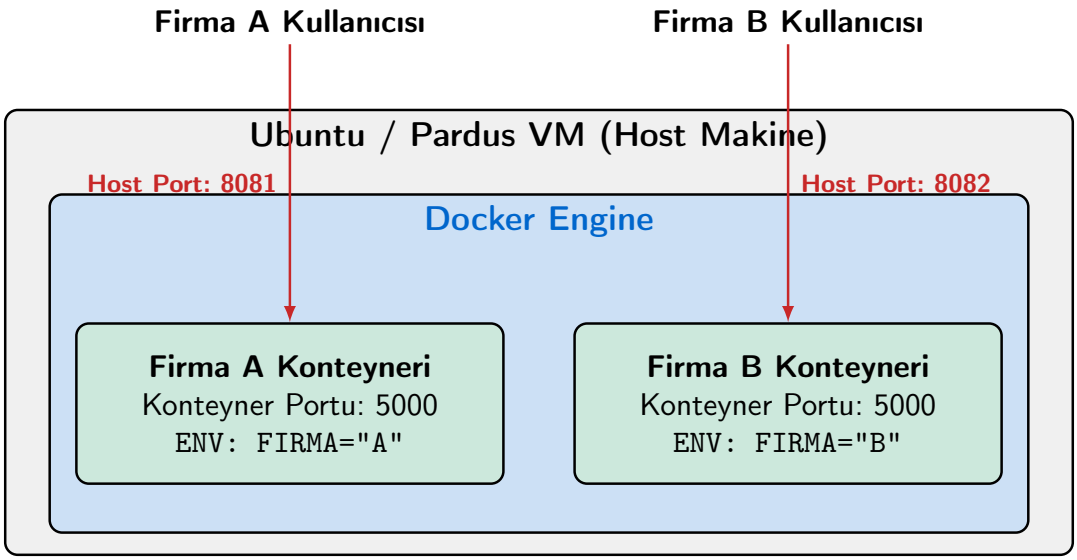
- Uygulamada en az **3 farklı route** bulunmalıdır. Örnek: `/`, `/durum`, `/iletisim`, `/paket`.
- Uygulama, en az bir değil tercihen birden fazla ortam değişkeni (`ENV`) kullanmalıdır.
- Çıktı olarak yalnızca düz metin değil, anlamlı **HTML içerik** üretilmelidir.
- Kod, öğrenci tarafından **özgün biçimde geliştirilmiş** olmalıdır.
- Demo sırasında öğrenci, kodun her bölümünü açıklayabilmelidir.

Özgünlük beklentisi:

- Sadece derste verilen en basit örnek kodun birebir kopyalanması yeterli değildir.
- Route isimleri, HTML düzeni, değişken adları ve uygulama içeriği mümkün olduğunca öğrenciye özgü olmalıdır.
- `app.py` dosyasına öğrencinin kendi yorumunu kattığı açık biçimde görülmelidir.

3. Sistem Mimarisi

Proje, aşağıdaki mantığa uygun olarak fiziksel veya sanal makine üzerinde çalışacaktır.



4. Adım Adım Geliştirme Rehberi

4.1. Adım 1: app.py Dosyasının Yazılması

Aşağıdaki ilk örnek, en temel mantığı gösterir:

```
from flask import Flask
import os

app = Flask(__name__)
firma = os.getenv("FIRMA", "Bilinmeyen")

@app.route('/')
def index():
    return f"<h1>{firma} paneline hosgeldiniz</h1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Ancak proje için öğrenciden bundan daha gelişmiş ve özgün bir yapı beklenmektedir. Aşağıdaki örnek, geliştirilmiş bir referans niteliğindedir:

```
from flask import Flask
import os
from datetime import datetime

app = Flask(__name__)

firma = os.getenv("FIRMA", "Bilinmeyen")
paket = os.getenv("PAKET", "Standart")
bolge = os.getenv("BOLGE", "TR")
surum = os.getenv("SURUM", "v1.0")
```

```
@app.route('/')
def home():
    return f"""
    <h1>{firma} Musteri Paneli</h1>
    <p>Paket: {paket}</p>
    <p>Bolge: {bolge}</p>
    <p>Surum: {surum}</p>
    <p>Bu uygulama Docker konteyneri icinde calismaktadir.</p>
    """

@app.route('/durum')
def durum():
    return f"""
    <h2>Sistem Durumu</h2>
    <ul>
        <li>Firma: {firma}</li>
        <li>Paket: {paket}</li>
        <li>Bolge: {bolge}</li>
        <li>Sunucu Saati: {datetime.now()}</li>
        <li>Servis Durumu: Aktif</li>
    </ul>
    """

@app.route('/paket')
def paket_bilgi():
    return f"""
    <h2>Paket Bilgisi</h2>
    <p>{firma} icin aktif paket: {paket}</p>
    """

@app.route('/iletisim')
def iletisim():
    return f"""
    <h2>Iletisim</h2>
    <p>{firma} destek birimine hos geldiniz.</p>
    <p>Bolgesel hizmet alani: {bolge}</p>
    """

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Not: Öğrencinin yukarıdaki örneği birebir kullanması tavsiye edilmez. Route isimlerinin, HTML içeriğinin, değişkenlerin ve uygulama mantığının kişiselleştirilmesi beklenmektedir.

4.2. Adım 2: Dockerfile Oluşturulması

Aşağıdaki Dockerfile örneği temel bir Flask uygulamasını çalıştırmak için yeterlidir:

```
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
CMD ["python", "app.py"]
```

İsteyen öğrenciler daha profesyonel bir yapı için `requirements.txt` da kullanabilir.

4.3. Adım 3: İmajın Derlenmesi ve Container'ların Çalıştırılması

Aşağıdaki komutlarla imaj derlenir ve iki farklı firma için iki ayrı container başlatılır:

```
# Imaji derleme
$ docker build -t saas .

# Firma A icin
$ docker run -d -p 8081:5000 -e FIRMA=A -e PAKET=Premium --name A saas

# Firma B icin
$ docker run -d -p 8082:5000 -e FIRMA=B -e PAKET=Standart --name B saas
```

4.4. Adım 4: Kontrol ve Test

Aşağıdaki komutların doğru kullanılması beklenir:

```
$ docker images
$ docker ps
$ docker logs A
$ docker logs B
```

Tarayıcıdan şu adresler test edilmelidir:

```
http://<IP_ADRESI>:8081
http://<IP_ADRESI>:8081/durum
http://<IP_ADRESI>:8082
http://<IP_ADRESI>:8082/paket
```

5. Teslim Kuralları

1. **Tek PDF Dosyası:** `app.py`, `Dockerfile`, terminal ekran görüntüleri ve tarayıcı çıktıları tek bir PDF içinde toplanmalıdır.
2. **O'Learn Zorunluluğu:** Teslim yalnızca O'Learn sistemi üzerinden yapılacaktır.
3. **Canlı Demo Zorunluluğu:** Öğrenci, projesini laboratuvarda çalıştırarak hocaya göstermek zorundadır.
4. **Özgün Kod Zorunluluğu:** Kodlar, route yapıları ve HTML çıktıları mümkün olduğunca özgün olmalıdır.
5. **Açıklanabilirlik:** Öğrenci, demo sırasında kendi yazdığı kodu açıklayabilmelidir.

Önemli Not: Çalışmayan, açıklanamayan veya kopya olduğu kanaati oluşan projeler değerlendirme dışı bırakılabilir.

6. Notlandırma (100 Puan)

Değerlendirme Seviyeleri

90-100: Tam çalışır, özgün, açıklanabilir sistem

Kriter	Açıklama	Puan
Dockerfile	Reçetenin doğru ve çalışır biçimde yapılandırılması	15 Puan
ENV Kullanımı	Çevre değişkeni ile dinamik veri alınması	20 Puan
Port Yönetimi	Multi-container yapısında doğru port yönlendirmesi	20 Puan
Network Erişimi	Tarayıcı üzerinden başarılı erişim sağlanması	15 Puan
app.py Özgünlüğü	Route yapısı, HTML içeriği, geliştirme düzeyi ve açıklanabilirlik	30 Puan

70-89: Çalışır ama sınırlı geliştirme

50-69: Temel çalışır, eksikler var

0-49: Çalışmıyor veya açıklanamıyor

7. Kopya Tespit Sistemi (Hocaya Özel)

Bu projede değerlendirme yalnızca sonucun çalışmasına göre değil, öğrencinin kodu üretme ve açıklama kapasitesine göre yapılacaktır.

Otomatik şüphe göstergeleri:

- Farklı öğrencilerde aynı route isimlerinin ve aynı sıra düzeninin görülmesi.
- HTML çıktı metinlerinin karakter düzeyinde birebir aynı olması.
- ENV değişken isimlerinin, değişken sıralarının ve Dockerfile yapısının tamamen aynı olması.
- Kod içinde öğrenciye ait açıklama, yorum veya kişisel özelleştirme bulunmaması.
- Tüm projenin tek bir örnek kaynaktan birebir çoğaltıldığı izlenimi vermesi.

Hızlı benzerlik metriği:

$$\text{Similarity} = \frac{\text{Ortak Satır Sayısı}}{\text{Toplam Satır Sayısı}}$$

Eğer yaklaşık olarak

$$\text{Similarity} > 0.7$$

ise proje **kopya şüphesi** taşıyabilir.

Davranışsal kopya göstergeleri:

- Öğrenci `os.getenv` fonksiyonunun ne yaptığını açıklayamıyorsa,
- Yeni bir route ekleyemiyorsa,
- `-p 8081:5000` ifadesini yorumlayamıyorsa,
- Kod içinde küçük bir değişiklik yapması istendiğinde yapamıyorsa,
- Hangi dosyanın ne işe yaradığını karıştırıyorsa proje şüphelidir.

Uygulama kuralı: Şüpheli durumda öğrenciye canlı değişiklik yaptırılır. Yapamıyorsa proje geçersiz sayılabilir.

8. Demo Sırasında Sorulacak Ayırt Edici Sorular

Bu sorular, ezber yapan veya proje mantığını anlamadan kopyalayan öğrenciyi ayırt etmek için etkilidir:

- **ENV değişkeni nedir? Docker içine dışarıdan nasıl aktarılır?**
- `-p 8081:5000` **ifadesinin sol ve sağ tarafı tam olarak neyi gösterir?**
- **Aynı host portunu iki farklı container neden aynı anda kullanamaz?**
- **Flask içinde** `host='0.0.0.0'` **neden yazılır? Yazılmazsa ne olur?**
- `WORKDIR` **komutu Dockerfile içinde ne işe yarar?**
- **Container silinirse uygulama, log ve dosya durumu ne olur?**
- `docker ps` **ile hangi bilgileri görürüz?**
- **Şimdi koda yeni bir route ekle ve tekrar çalıştır.**

- **Runtime sırasında `FIRMA` değerini nerede ve nasıl veriyorsun?**
- **Neden doğrudan Flask kullandık; neden Nginx ya da başka bir yapı kullanmadık?**

ALTIN SORU (Çok Etkili)

"Şu an 2 müşteri için 2 container ayağa kaldırdın. Bu sistemi 100 müşteriye satacak olsaydın nasıl ölçeklerdin? 100 defa `docker run` mı yazardın, yoksa daha doğru bir yöntem mi kurardın?"

9. Hızlı Kontrol Checklist'i

1 dakikalık teknik tarama:

- ☐ Container'lar gerçekten çalışıyor mu? (`docker ps`)
- ☐ İki farklı host portu aktif olarak doğru bağlanmış mı?
- ☐ ENV parametresi iki farklı müşteri için farklı çıktı üretiyor mu?
- ☐ En az 3 route tanımlanmış mı ve hepsi çalışıyor mu?

2 dakikalık derin tarama:

- ☐ Kod ve HTML yapısı diğer öğrencilerden farklı mı?
- ☐ Öğrenci route mantığını açıklayabiliyor mu?
- ☐ Öğrenci küçük değişiklikleri anında yapabiliyor mu?
- ☐ Kod gerçekten öğrencinin kendi emeğini yansıtıyor mu?

Kırmızı bayraklar:

- ☐ Aynı sınıfta aşırı benzer proje yapıları
- ☐ Öğrencinin kodu okuyup açıklayamaması
- ☐ ENV mantığının bilinmemesi
- ☐ Route, HTML ve değişken adlarının başka projelerle birebir eşleşmesi

Başarı = Teknik Çalışma + Anlatım + Özgünlük